

Project
REF: Resource Evaluation and Feedback DSL
Project Assignment P2: DSL and Code
Generation

ENORM

Master in Informatics Engineering - 2025/2026

Porto, April 15, 2026

Alexandre Bragança (ATB),

Rafael Gonçalves (RCC)

Version 2, 2026-04-10

REVISION HISTORY

Revision	Date	Author(s)	Description
1	2026-04-04	atb	Initial working version
2	2026-04-10	atb	Updates some items in activity 6

CONTENTS

1	Context	1
1.1	The Problem	1
1.2	REF: A DSL for Resource Evaluation and Feedback	2
1.2.1	Simple, Intuitive and Easier to Learn	2
1.2.2	Strongly Typed	3
1.2.3	Complete	4
1.2.4	Flexible and Extensible	4
1.2.5	Evolvable	4
1.3	DSL Engineering	4
1.3.1	Domain Engineering	5
1.3.2	Application Engineering	5
2	Organization and Process	9
2.1	Teams and Tools	9
2.2	Repositories	10
2.3	Reports	11
2.4	Process	11
2.5	Submission	14
3	Domain Engineering	15
3.1	Concrete Syntax	15
3.1.1	Graphical Notation	16
3.1.2	Textual Notation	16
3.2	Common Features of Applications	17
3.3	Commonality and Variability in the Code	18
3.4	Code Generation	19
4	Terms and conditions	21
4.1	Submission	21

4.2	Report	21
4.3	Deadline	21
4.4	Feedback	21
4.5	Evaluation	21
4.6	Grading	22
4.6.1	Grading Criteria	22
4.7	Code of Conduct	23

LIST OF FIGURES

1.1	A simple textual syntax (from http://www.wcse.org/WCSE_2017/041.pdf	3
1.2	An Example of Graphical syntax based on BPMN (from https://pdfs.semanticscholar.org/ca5/ee06b4ec40099ae5ee7f803ef10a59b2fe25.pdf	3
1.3	Domain Engineering.	5
1.4	Application Engineering.	6
3.1	Metamodel for Hello World DSL.	15
3.2	Graphical notation for concept Web.	16
3.3	Graphical notation for concept Console.	16

Chapter

1

CONTEXT

This document describes the second part (P2) of the ENORM project, which overall goal is the exploratory development of tools in the context of a Software Product Line. Specially, the aim is to develop a domain-specific language and some other supporting tools. This chapter presents an overview of the ENORM project, and is equal to the Context Chapter of P1. Chapter 2 presents some guidelines on the organization and procedures to be used for P2. Chapter 3 presents details of the development of the core assets of the product line. Finally, Chapter 4 presents the terms and conditions of the project.

1.1 The Problem

The ENORM project considers the simplified case of a software company that is specialized in the development of web based solutions. **The company has developed several web solutions and, through the years it has identify some commonalities in those solutions.** Resulting from this constact it has decided to explore those **using a software product line (SPL) approach.** In the context of the new SPL approach, it will also explore the use of a DSL. **The ENORM project regards the exploration of this DSL.**

The great majority of the **web solutions developed by the company include some sort of user evaluation or feedback features.** For instance, in e-commerce applications it is very common to allow users to evaluate the products they buy. Similarly, in web applications were users post content (such as videos or photos) it is usual to give users the possibility to comment on such posts.

One of the technical aspects of **these solutions is that they rely on a REST architecture,** and the backend is composed of several REST API based microservices that are very similar. The first approach of the company was to use a DSL to help developers create these APIs. As such, the company developed a DSL that helped developers generate a significant part of the

code¹.

The results of this initial DSL were promising. However, the DSL is technical and aimed at software engineers. The company now wishes **to explore a more high level DSL, one that can be used by subject matter experts (SME)**. The ENORM project is this next phase of the exploration of this approach. **The main objectives are to build a MVP of a DSL that:**

- can generate at least 90% of the code required to implement the backend microservices that support the users feedback and evaluation of the web applications produced by the company;
- can be used by subject matter experts as they describe/specify their needs in this context;
- supports both textual and graphical syntaxes;

The next section will present more specific aims of the project.

1.2 REF: A DSL for Resource Evaluation and Feedback

The goal for the project is to produce a DSL called REF (Resource Evaluation and Feedback). As described before, this DSL is inspired by previous work of the company developing a DSL for REST API creation. The previous DSL targeted the software engineers of the company, reducing the time they spent coding very similar REST web services. This experience was successful, and now the company wants evolve even further the usage of DSLs to new users. This time it is targeting the subject matter experts (SME). These are usually professionals from the clients of the company that know very well the business domain, but do not have the technical skills for developing the solutions, they are not software engineers. **The aim is now to create a new DSL that will be used by these SME to specify their domain applications.** Based on these application specifications done with the new DSL, it should be possible to generate the majority of the code. The software engineers that previously developed the solutions will be trained to be able create and maintain the DSL as well as coding the possible 10% of code that will not be generated by the DSL.

The following sections detail some main characteristics that the DSL should support.

1.2.1 Simple, Intuitive and Easier to Learn

The DSL should be simple and easier to learn. It should be as much intuitive as possible for the subject matter expert.

The company made some research about DSL syntaxes and found some examples for other domains. For instance, Figure 1.1 and Figure 1.2 present examples of textual and graphical syntaxes of DSLs aimed at modeling Extract-Transform-Load (ETL) processes.

¹A solution similar to the one available in <https://github.com/HachimHassani/Rest-DSL>

The textual syntax of the new DSL should be simple and easier to learn. **It should be, as much as possible, close to how a person would describe the application in natural language.**

Figure 1.1: A simple textual syntax (from http://www.wcse.org/WCSE_2017/041.pdf)

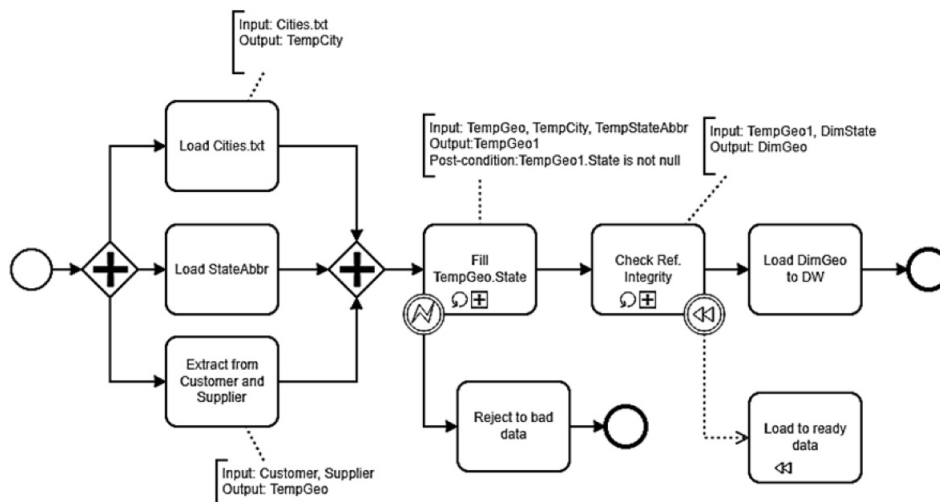
```

jobname:loadFileandDataDraft
author: Sunisa Junsawang
version:1.0
description:loadFileandDataDraft
step: FileLoadtoTableA
type: loadfile
source: input_file/test.file
format: header=yes , footer=no , delimiter=pipe , quote=no
target: tableA (append)
end
step: deletetoTableC
type: delete
target: tableC
condition: att3 > '2000'
end

```

Regarding the graphical notation, it should also be simple and **use intuitive graphical elements**. It should avoid cluttering the diagrams with too much detail. **A common way to avoid cluttering is to display detail as text in property windows.**

Figure 1.2: An Example of Graphical syntax based on BPMN (from <https://pdfs.semanticscholar.org/caf5/ee06b4ec40099ae5ee7f803ef10a59b2fe25.pdf>)



1.2.2 Strongly Typed

The DSL should be strongly typed, in order to help its users (SME) to avoid specification errors when writing DSL programs. Basically, the idea is that **when the user ends the creation of**

a DSL program, that program/specification should always be a valid specification so that the generated code is also always valid.

1.2.3 Complete

A domain usually includes not only structural but also behavioral elements. They take form as rules or processes that apply to the domain. We expect that this also applies to this REF domain. Therefore, it is expected that **the new DSL includes support to model both structural and behavioral aspects of the domain.**

1.2.4 Flexible and Extensible

Although the DSL should be simple and provide the most common features for the resource evaluation and feedback domain, it is possible that these functionalities do not cover all the cases. Therefore, the DSL should be extensible and flexible by, for instance, **providing an extensibility mechanism that the SME can use to add new operators/functions in the DSL that are then implemented by a software engineer in the base programming language (e.g., java).**

1.2.5 Evolvable

The DSL should be designed to support its evolution. When working with DSLs their evolution is a mandatory feature. If a DSL is to be successfully used it must be designed up-front thinking that it will, sooner or later, require changes. Therefore, **it is required to prepare the DSL with the necessary tools to support the migration of DSL programs to new versions of the DSL.**

1.3 DSL Engineering

The development process is divided in two levels: **domain engineering** and **application engineering**. These levels of engineering, although connected, are usually performed by very different roles. Domain engineering requires someone that is expert in building domain tools, such as domain-specific languages. Application engineering requires another role, someone which will use the previous domain tools (e.g., DSL) to build applications in the domain. This requires knowledge in the domain, but not on building domain tools (e.g., building DSLs), usually this is the role of the subject matter expert. For the ENORM project, you should have to play the two roles.

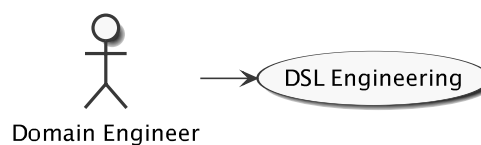
1.3.1 Domain Engineering

Domain engineering is responsible for developing reusable artifacts, i.e., domain-specific artifacts that are reused when building domain applications. In this particular case, the goal is to develop a **domain-specific language and supporting processes/tools** (e.g., model validations, model transformations and code generation) for building applications (in this specific case, backend services of the applications) (see Figure 1.3). The artifacts that result from domain engineering are then used at the level of **application engineering** that, obviously, in this case, will use the REF DSL to model and generate backend services that support resource evaluation and feedback processes of applications for the domain (see Figure 1.4).

For this part of the project (P1), the major goal is to design the metamodel of the new REF language. This includes not only the structure of concepts (its properties and relations) but also model validations and model transformations (refactorings). To construct the metamodel, domain knowledge must be extracted from studying documents and talking to the subject matter experts (in this case, teachers will play this role). With this knowledge, a design for the metamodel of the new language must be proposed that tries to comply with the previous presented aims and tries to support the specific requirements presented in section 3. You should also comply with the instructions of the laboratory class teacher that, in some way, will play the roles of subject matter expert and product owner.

For this part of the project (P1), it is not necessary to develop the full DSL! You are only required to build the parts described previously. It is not required to design and build the concrete syntax of the DSL, only if the tool you use requires it. If so, you may develop a "raw" syntax only to support the aims of P1. Therefore, **for P1, you are not required to design the concrete syntax of the DSL and you are not required to generate code.** It is, however, also required to develop the tasks described in the next section (Section 1.3.2), that are related to application engineering.

Figure 1.3: Domain Engineering.

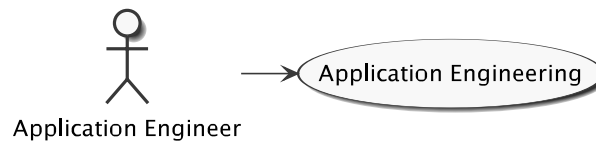


1.3.2 Application Engineering

Application engineers reuse the tools developed in domain engineering (e.g., DSLs) to create domain applications, as depicted in Figure 1.4. **When the reusable tool is a DSL the application engineer role can potentially be played by a subject matter expert.**

Since the goal of this project is to evaluate the feasibility of this approach in an MVP, it was decided to replicate three well-known cases of REF. Therefore, you should demonstrate the

Figure 1.4: Application Engineering.



solution with the following cases, that are briefly described:

- **Case 1: Youtube:** In this case the evaluated resources are user videos. Users can comment other users' videos and 'rate' them using thumbs up or down. Also, comments may start threads. Comments are usually sorted by date or "value". Users may report comments that violate community rules;
- **Case 2: Amazon:** In this case the evaluated resources are the products that users buy. Users can evaluate products. Usually users that evaluate products are also buyers, but this rule is not mandatory. Users classify the products using a five star grading scale. The evaluation includes text but may also include photos and videos. There is also a verification process for evaluations that can be manual or automatic;
- **Case 3: Reddit:** In this case the resource is similar to a discussion forum that has a specific topic. Users can post in the forum. There are different types of posts. When users reply to posts they create discussion threads. Users can evaluate posts and comments by "upvoting" and "downvoting" on them. Users may report on posts and comments. Each topic has its own rules and moderators. The platform supports a mechanism to automate some related processes.

The developed DSL must support all these three scenarios. It is up to you to research the specific requirements and features of these 3 scenarios. The previous description is only a high level overview of features.

The company is expecting that the designed DSL includes/supports the following requirements:

- **Structure:** it should be possible to **model the diverse properties of resources**;
- **Authorization:** it should be possible to **model the diverse users/actors that interact with the application and what operations can they perform**;
- **Behaviour:** it should be possible to **model the diverse processes related to the actions on the resource**, such as moderation processes and automatic validations. Reddit seems particularly powerful in this regard, and the company is expecting support for something similar to Reddit's **Automations**.

Remember that the goal of this MVP is to initially support at least these scenarios. It does not have to mimic exactly the three applications, but it must support similar applications. Also, the idea is that **the DSL should support its evolution, as new scenarios are identified and the DSL is updated to support them.**

The modeled scenarios should provide sufficient data to demonstrate and test the major features of the DSL regarding its structure, validations and refactorings.

ORGANIZATION AND PROCESS

This ENORM project is divided in three parts or components:

- **P1 Modeling**

- This is the first part of the development of the DSL, focusing on modeling
- It starts at week 2 and its planned to end by the middle of the semester (at the end of week 7)
- It is the part that is described in this document
- This is a team component with individual activities and individual grading

- **P2 DSL and Code Generation**

- This is the second part of the development of the DSL, focusing on the DSL and code generation
- It is planned to start at the beginning of week 8 and ending at the end week 12
- It will be described in another document
- This is a team component with individual activities and individual grading

This document only describes the requirements for **P2 DSL and Code Generation**.

2.1 Teams and Tools

Students should form teams of three elements (exceptionally teams can have less members, but only with the teacher's acceptance).

Each member of the team is responsible for the development of the solution using a specific tool. You should **check the technical requirements for the installation and usage of the tool**, e.g., some tools may have specific requirements.

Each team member should select a unique tool from the following ones. All members of the same team should select different tools.

- **MPS - Meta Programming System**

- Site: <https://www.jetbrains.com/mps/>
- Documentation: <https://www.jetbrains.com/help/mps/mps-user-s-guide.html>
- Tutorial video: <https://youtu.be/T1RErEvgRc>

- **Xtext**

- Site: <https://www.eclipse.org/Xtext/>
- Documentation: <https://www.eclipse.org/Xtext/documentation/index.html>
- Demonstration/Tutorial video: <https://vimeo.com/138873446>

- **Sirius**

- Site: <https://www.eclipse.org/sirius/>
- Documentation: <https://www.eclipse.org/sirius/doc/>
- Advanced documentation available at <https://wiki.eclipse.org/Sirius/Tutorials/AdvancedTutorial>
- Tutorial videos:
 - * Part 1 <https://youtu.be/uKBGVwmRzi0>
 - * Part 2 <https://youtu.be/3rnBmG4DN4w>
 - * Part 3 <https://youtu.be/h0U62wf3bYs>
 - * Part 4 https://youtu.be/M1RdRsLic_k
 - * Part 5 <https://youtu.be/iHtmL-PKT8c>

Attention for Sirius and Xtext Tools: These tools are based on EMF. They can use a metamodel created as an ecore metamodel, as described in the following tutorial <https://www.vogella.com/tutorials/EclipseEMF/article.html>. Therefore, in the case of these tools, you may follow this tutorial to learn how to create ecore metamodels (or follow the demonstrations from the lectures)

The students may select other tools but only with the teacher's acceptance.

2.2 Repositories

Each team will have access to a team repository that should be used to develop the two team components: P1 Modeling and P2 DSL and Code generation. Further instructions will be available in the team repository.

2.3 Reports

For each part of the project (P1 and P2) there will be a report to be produced. All reports are produced inside the repositories using Markdown. Further instructions related to the reports will be available in the repositories.

For this part of the project (P2), the report is included in the team repository. However, the report is further divided in two parts: a team part and individual parts. Team and Individual parts are to be developed and reported according to instructions in Section 2.4 and also specific teacher's indications.

2.4 Process

The proposed process for P2 DSL and Code Generation is structured in the following activities. Usually they should be developed in a sequential way, for instance, one in each week. But this organization is not mandatory, the team should organize the work as best fit for their needs.

- Activity 1: **Design Concrete Syntax for the DSL**
 - The overall goal for this activity is to design the **notations for using in specific editors for models of the language (either graphical or textual)**
 - **Try to use notations that are, as much as possible, intuitive for a domain expert** (for instance, use a textual notation that is close to natural language)
 - You should continue using the metamodel and all the work developed for project P1 but **you can make adaptations if required (but you should justify them)**
 - Team tasks:
 - * Design a graphical syntax for the DSL
 - For each metamodel element specify how it should be graphically represented in a graphical editor
 - * Design a textual syntax for the DSL
 - For each metamodel element specify how it should be textually represented in a text editor
 - * Add to the report (team section) the previous designs
 - Individual tasks:
 - * Implement the designed notations in the selected tool
 - * Be aware that Sirius will support the graphical notation and MPS and Xtext will support the textual notation. So, each tool must only support one notation.

- * Add to the report (individual section) a description of the implementation and possible adaptations in the design to meet the tool specific requirements
- Activity 2: **Specify Common Features for Applications of the Domain**
 - Your DSL will use a generation approach (not an interpreted approach). Therefore, **the DSL will need to generate the code that supports the REST services of the App backend.**
 - The goal for this activity, and the next one, is to develop the backend of applications (i.e., REF applications) of the domain (i.e., implement the REF structure and logic/behavior) in, as much as possible, a very similar way, such as, building the first one and then copy-and-paste to build the others
 - In this activity you should select the language you are going to use to implement the REF backend, the architecture of the generated code, and common features of the generated code. For instance, if you are going to implement the system in Java you need to explore how you will design the architecture of the code you will generate from the DSL. You also need to identify the common features of this code (i.e., the supporting code that you will always need, no matter what the domain experts models in the DSL). For instance, if your DSL uses the concept of Post maybe you will always need a Java class that implements this Post concept
 - **Important (new requirement)** Take also into consideration that certain aspects of the generated code may require (or should require) support for manual adaptations/extensions from the user. For instance, in P1, one of the characteristics of the language was that it should "...providing an extensibility mechanism that the SME can use to add new operators/functions in the DSL that are then implemented by a software engineer in the base programming language (e.g., java)". Therefore, we should **add support in the generated code for the user to add logic using the base language in a way that is integrated with the generated code.** Consider this requirement in P2 while designing the code generation.
 - Team tasks:
 - * Add to the report (team section) a description of the common features for the applications, for instance, the language and frameworks to use, the architecture to use, the common code to be present, etc.
- Activity 3: **Implement Prototypes of Applications of the Domain**
 - Individual tasks:
 - * Using the decisions of the previous activity, each student should try to develop an example application in the domain (i.e., the code in the selected language that is able to execute a REF backend following the previous design decisions).

- * You may use the examples of Activity 7 of P1 as inspiration for the example code of the prototypes to be implemented
 - * This manually implemented code will be used in the next activity as a way to identify the "type of" code that needs to be generated by the DSL.
- **Activity 4: Identify Commonality and Variability in the Code**
 - Team tasks:
 - * The team should analyze the code of all the prototypes to find common and variable "parts"
 - * The team should relate the variable parts in the code to parts of the metamodel that capture those specifics
 - * All the parts of the metamodel should be mapped to some variable part of the code (or other artifacts of the applications)
 - Note that some metamodel elements can be mapped to code that must (or can) be manually completed or extended. See the new requirement introduced in Activity 2.
 - One way to address this problem could be to enable the user to "extend" the generated code to overcome some limitations of the DSL
 - * Add to the report (team section) the previous findings
 - **Activity 5: Design and Implement Code Generation**
 - Team tasks:
 - * Design the templates/rules for code generation
 - * Common parts of the code are always generated. Specify the common parts
 - * Variable parts of the code are generated based on the contents of the models. Specify the variable parts and state when and how they are generated based on model values
 - * Add to the report (team section) a description of the previous findings
 - Individual tasks:
 - * Each student should implement the code generation templates/rules in their tool
 - * All the tools should be able to generate all the applications of the domain
 - **Activity 6: Generate Applications**
 - Individual tasks:

- * Each student should be able to generate the code for the three REF models (applications) described in Activity 7 of P1 using is/her tool
 - * You also should include the possibility of manually adding code, as described in the new requirement presented in Activity 2.
 - * Each student should test the modeling and code generation of, at least, the previous described modeling scenarios
 - * Add to the report (individual section) the possible issues found during tests
 - * Explore how your tool can support model evolution, i.e., if you change the DSL metamodel how to migrate previous models to the new DSL version? Report a proposal to solve this problem. Demonstrate your solution.
- Team tasks:
- * The code generated by the different tools should be compatible, i.e., it should be possible to mix/combine the code generated from DSLs implemented in different tools
 - * The team should test the combination of code generated by different tools
 - * Add to the report (team section) the possible issues found during tests

2.5 Submission

By the end of week 12 (specific date to be announced) **each element of the team should submit a copy of the team repository in Moodle** (link to be announced).

Chapter

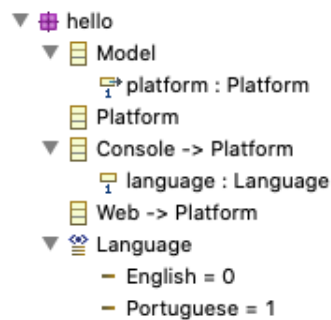
3

DOMAIN ENGINEERING

To help the fulfillment of all the activities presented in Section 2.4 some hints and guidelines are presented in this Chapter.

To exemplify we will consider the following metamodel for a very simple DSL for modeling hello world applications.

Figure 3.1: Metamodel for Hello World DSL.



As we can see, the metamodel is very simple. Basically, the modeler can select between Console or Web application. For Console, the modeler can then specify the language used, Portuguese or English.

3.1 Concrete Syntax

To design each concrete syntax try to find the notation that best fits each element of the metamodel. For graphical tools, this can be shapes, lines and text with specific formatting. For textual tools, basically we need to specify a grammar.

3.1.1 Graphical Notation

If we use the simple example of the Hello World DSL, we could define only a very simple box for Console as well as Web. Since both of these concepts do not have names, we could distinguish between them using the name of the concept at the top of the box. In the case of Console, the attribute "language" could be depicted inside the box.

Figure 3.2: Graphical notation for concept Web.

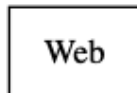
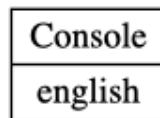


Figure 3.3: Graphical notation for concept Console.



Remember that this is a very simple DSL. Usually a DSL will require also different types of shapes for different concepts. Also, remember that references (that are not containment references) between concepts will result in links (or lines) in the graphical notation. But concepts may also be represented by lines. For instance, in the case of the UML metamodel, an include is represented in the metamodel by the concept Include (a class) but it is represented as a line/arrow in the diagrams. Usually, containment references between concepts result in contained elements being visually represented inside their containers. This is not very different from a field that is represented inside the container class in UML class diagrams, for instance. In the example of the hello world DSL we see also something similar in the language attribute that is represented inside the console box.

3.1.2 Textual Notation

To completely specify the textual notation of a DSL we need to specify the grammar of the language, i.e., the rules for non-terminals and the terminals of the language.

For the hello world DSL this could result in something like in the following listing.

Grammar for the hello world DSL

```
1 Model :  
2     'Model'  
3     '{'  
4         'platform' Platform  
5     '}' ;  
6  
7 Platform :
```



```
8      Console | Web;
9
10 Console:
11     'Console'
12     '{'
13         'language' Language
14     '}' ;
15
16 Web:
17     'Web'
18     ;
19
20 Language:
21     'English' | 'Portuguese';
```

In the listing, the red words are terminal symbols. The other words are non-terminals. For all non-terminals we need to have rules. Each rule specifies how a non-terminal is composed by other elements, terminals or non-terminals. To be noticed that this is one possible example of textual notation. We could define several others for the same language. Also note that the example presented is highly inspired by the notation used by default by the Xtext framework. **This may not be a proper syntax for non-technical users, since it is based on a syntax similar to programming languages such as C++ and Java, using, for instance, {} blocks.**

The following listing presents a possible representation of a model following the previous textual notation (i.e., grammar).

Possible program for the hello world DSL

```
1 Model
2 {
3     platform Console
4     {
5         language Portuguese
6     }
7 }
```

3.2 Common Features of Applications

Usually, the final goal of a DSL is to be able to generate applications for a specific domain. In order to be able to do so we need to make some decisions regarding the architecture and also technical options for our generated applications. For instance, what programming language will be used? what frameworks? what kind of applications will be generated? what dbms will be used? etc. We will need to answer questions like these in order to define the base architecture and platform for the generated applications. After that, it is possible to envision how the applications in the domain can be codified.

In the simple case of the hello world DSL the idea is to generate simple hello world applications. As the metamodel implies, these applications can be console applications or web

applications. One of the questions in this case is what programming language to use. We could select Java. For the web applications we only need to use HTML. Therefore, for this simple example, these two decisions are sufficient to define the base "platform" for the generated code/applications.

3.3 Commonality and Variability in the Code

One of the best ways to identify what code (or other artifacts) to generate is to implement sample applications in the domain (or study the code of existing applications). We should execute this task in the context of the decisions made previously regarding the common features. For the sample hello world DSL we could imagine the following 3 examples of generated code.

Java1

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("Hello World");
4     }
5 }
```

Java2

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("Ola Mundo");
4     }
5 }
```

Html

```
1 <!DOCTYPE html>
2 <html>
3     <head>
4         <title>Hello World</title>
5     </head>
6     <body>
7         <p>Hello World.</p>
8     </body>
9 </html>
```

After implementing the code it is possible to identify parts of the code that are common and other that are variable.

Common parts are parts that are always present in the code. In our examples we do not have common parts for all the applications.

Platform variability However, we have parts of code that are common for the console Platform. These are the lines 1, 2, 4 and 5 of the java code. These lines of code should be generated when the model includes the Console element. In the case the model includes the Web element, the HTML code should be generated. We see then that Console should generate

java and Web should generate HTML. So, each subclass of Platform will generate different files.

Console.language variability In the case of the java code there is also line 3 that is different between applications. In one case it outputs the message in English and in the other case in Portuguese. This is because it is related to the "language" attribute of the Console concept that can have one of those two values.

3.4 Code Generation

Code generation it is usually based on the use of templates. Templates are basically "incomplete" texts that have placeholders that are replaced in order to obtain the resulting "complete" text. Rules are then defined to apply these templates in the context of specific elements of models that will be used to fill the placeholders in order to generate the final code/text.

For the hello world example, and based in the previous findings, we could define the following rules and templates.

Rule: Platform=Web The code to be generated should be the one in the following listing.

Template 1

```
1<!DOCTYPE html>
2<html>
3  <head>
4    <title>Hello World</title>
5  </head>
6  <body>
7    <p>Hello World.</p>
8  </body>
9</html>
```

Rule: Platform=Console The code to be generated should be the one in the following listing.

Template 2

```
1public class Main {
2  public static void main(String[] args) {
3    //[if Console.language=english]
4    System.out.println("Hello World");
5    //[else]
6    System.out.println("Ola Mundo");
7    //[endif]
8  }
9}
```

Note that the lines 3, 5, and 7 contain meta-code (i.e., macros) for the template engine, in order to replace the "placeholders" with the correct code. In this case we see a special case of an **if then else** "placeholder". This code verifies the value for the attribute "language" of the

Console element and will include in the final text the line 4 or the line 6, therefore including the correct code in accordance with the model.

TERMS AND CONDITIONS

4.1 Submission

All the developed artifacts of the project should be included in your team repository (according to instructions).

For the Moodle submission a file containing a copy of the repository must be download from Github and submitted in Moodle using a specific link (of Moodle, that will be provided later).

4.2 Report

All the documentation should be included in the repository, using Markdown, and specific instructions included in the repository.

4.3 Deadline

Until 23:59 of the Sunday, May 31

4.4 Feedback

Feedback can be requested to the teacher at any moment, during the PL classes.

4.5 Evaluation

Evaluation for this part of the project will occur at the end of the semester, usually during lab classes, during weeks 13 to 15.

Each Student must be prepared to explain, discuss, operate and demonstrate his/her individual tasks and goals as well as the team's tasks and goals. Further instructions on evaluation sessions will be provided.

Teachers may provide official documents attesting the presence of the student in an exam (only if previously requested by the student).

4.6 Grading

The weight of each part of the project will be provided in another document.

The evaluation of this project component has 2 parts:

- **Individual (60%).** This is specific to each team member and refers to his/her individual tasks and goals, in accordance to the performance of the student and his/her knowledge of the topics;
- **Team (40%).** This grade is related to the team's tasks and goals. It may be different for each team member, in accordance to the performance of the student and his/her knowledge of the topics;

4.6.1 Grading Criteria

A qualitative performance scale can be used by the students so that they can be aware of their own performance. This scale will also be used by teacher to grade each goal or task (described in Section 2.4)

Each task/goal can be graded following the next scale and the justification for each level:

- **0- nothing.** No submission or irrelevant;
- **1- tentative.** Do not achieve any of the requirements;
- **2- insufficient.** Achieves only a subset of the requirements, demonstrates insufficient knowledge of the evaluation subjects;
- **3- acceptable.** Achieves the requirements but with major faults, demonstrates acceptable knowledge of the evaluation subjects;
- **4- good.** Achieves the requirements with minor faults, justifies options, demonstrates good knowledge of the evaluation subjects;
- **5- excellent.** Achieves completely the requirements, justifies options, analyses alternatives in a quantified and rigorous way, demonstrates outstanding knowledge of the evaluation subjects;

The final grade for the project component results from the combination of the grades for all the tasks/goals (individual and team) and then converted to a 0-20 scale. Although in the scale only discrete values are presented (to facilitate the self-assessment by the student and also the feedback), teachers may use all the possible numeric values in the range of the scale 0-5.

4.7 Code of Conduct

- **Rules** cf. "Regulamento Disciplinar dos Estudantes do IPP"
- **Clearly identify sources and authorship** Code or any other material from external sources must be clearly identified in the code itself and in the reports, indicating the author or source.
- Any **misappropriation of authorship** (or collusion in the developed work) **will be sanctioned** according to the regulation, e.g., the evaluation component will be canceled. Also, the incident will be reported to the President of ISEP.
- **IA Tools usage** It is acceptable the usage of IA tools as a "research tool". However, it is expected (and mandatory) that all submitted artifacts are the student's authorship.